# Smart Campus Sustainability

A PROJECT REPORT

Submitted by
Yajat M Rawal-RA2511030040010
S Surendar -RA2511030040005
Surya Swaminathan-RA2511030040008
Shivraj Singh-RA2511030040021

Under the guidance of
Dr Arun Nehru J
(Associate Professor, Department of CSE
Emerging Technologies)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY
in

COMPUTER SCIENCE AND ENGINEERING with specialization in Cyber Security

of

FACULTY OF ENGINEERING AND TECHNOLOGY
of




DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
(EMERGING TECHNOLOGIES)
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
VADAPALANI CAMPUS

OCTOBER 2026

# SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Under Section 3 of UGC Act-1956)

BONAFIDE CERTIFICATE

Certfied that 21CSC203P-Advance programming practice report titled "**Smart Campus Sustainability and Carbon footprint monitoring System"** is the bonafide work of **Yajat M rawal-RA2511030040010,S SURENDAR-RA2511030040005,SURYA SWAMINATHAN-RA2511030040008,SHIVRAJ SINGH-RA2611030040021,**who carried the project under my supervision.Certified further,that to the best of my knowledge the work reported here in does not form an any other project report dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

GUIDE
Dr J Arun Nehru
Associate Professor
Dept. of CSE (Emerging Technologies)

HEAD OF THE DEPARTMENT
Dr. P. Chitra,
Professor
Dept. of CSE (Emerging Technologies)

# ACKNOWLEDGEMENT

We express our sincere gratitude to the management of SRM Institute of Science and Technology, Vadapalani Campus, for providing the facilities and an encouraging academic environment for this work.

We are deeply grateful to **Dr. P. Chitra**, Professor and Head of the Department of Computer Science and Engineering (Emerging Technologies), for the constant support and for creating opportunities to work on problems that matter to our own campus.

We owe special thanks to our guide, **Dr. Arun Nehru J**, Associate Professor, Department of CSE (Emerging Technologies), for the guidance, patient reviews and valuable suggestions at every stage of the project, from defining the carbon accounting boundary to evaluating the final system.

We also thank the faculty members and laboratory staff of the department, and the estate and facility teams whose descriptions of everyday campus operations shaped the alerting and work-order features.

Finally, we thank our families and friends for their encouragement throughout the course of this project.

Yajat M Rawal (RA2511030040010)
S Surendar (RA2511030040005)
Surya Swaminathan (RA2511030040008)
Shivraj Singh (RA2511030040021)

# ABSTRACT

Universities and colleges behave like small cities: they consume electricity, water and fuel at scale, generate waste and move thousands of people every day, yet most institutions measure their environmental impact only once a year from utility bills. Annual figures are too late and too coarse to change behaviour or operations, and faults such as leaking tanks or air-conditioning left running overnight go unnoticed for weeks.

This project designs and implements **EcoSphere AI**, a Smart Campus Sustainability and Carbon Footprint Monitoring System. Readings from smart meters and IoT sensors are ingested through an authenticated gateway API, validated, stored and converted into greenhouse-gas emissions by a carbon engine that follows the GHG Protocol (Scope 1, 2 and 3) using version-controlled emission factors such as the CEA grid factor. An analytics layer learns each building's normal load profile and raises alerts for after-hours consumption, demand spikes, night-time water leaks, offline or stuck meters and poor indoor ventilation. A regression model forecasts daily electricity demand from the academic calendar and temperature, a benchmarking module ranks buildings by Energy Performance Index, and a scenario model estimates savings from LED retrofits, rooftop solar, set-point changes, occupancy controls and electric buses before money is spent.

The system is built entirely with free and open-source software: Python, Django REST Framework, SQLite, NumPy and ReportLab on the server, and React, TypeScript, Recharts and Leaflet in the browser. Role-based dashboards serve management, the facility team and the student community, and a reporting module produces GHG inventory PDFs and CSV exports.

Evaluated on a 120-day simulated dataset for a ten-building campus of 5,850 people, the platform reported an annualised footprint of about 7,975 $tCO_2e$ (1.53 t per student), detected all six injected faults with no false alarms, and forecast daily demand with a mean absolute percentage error of 2.36%. A combined package of measures was estimated to avoid about 797 $tCO_2e$ per year with a payback of 3.1 years. The results show that continuous monitoring can turn sustainability from an annual report into a daily operating practice.



**Keywords:** smart campus, carbon footprint, GHG Protocol, IoT, anomaly detection, energy forecasting, Django, React, sustainability dashboard.

# TABLE OF CONTENTS


**ACKNOWLEDGEMENT** **i**
**ABSTRACT** **ii**
**1. INTRODUCTION** **1**
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope of the Project 1
1.5 Organisation of the Report 2
**2. LITERATURE SURVEY AND STANDARDS** **2**
2.1 Carbon Accounting Standards 2
2.2 Campus Sustainability Rating Systems 2
2.3 Existing Approaches 2
2.4 Analytical Techniques 3
**3. REQUIREMENTS ANALYSIS** **3**
3.1 Stakeholders and Roles 3
3.2 Functional Requirements 3
3.3 Non-Functional Requirements 4
3.4 Hardware and Software Requirements 4
**4. SYSTEM DESIGN** **5**
4.1 Layered Architecture 5
4.2 Data Flow 6
4.3 Database Design 7
4.4 API Design 8
4.5 Security Design 8
**5. METHODOLOGY AND ALGORITHMS** **8**
5.1 Carbon Accounting Method 8
5.2 Anomaly Detection 9
5.3 Demand Forecasting 9
5.4 Benchmarking and Scenario Modelling 10
**6. IMPLEMENTATION** **10**
6.1 Project Structure 10
6.2 Carbon Engine 10
6.3 IoT Ingestion and Simulation 11
6.4 User Interface 11
**7. RESULTS AND DISCUSSION** **12**
7.1 Campus Footprint 12
7.2 Energy Use Patterns 13
7.3 Fault Detection 14
7.4 Forecasting Accuracy 15
7.5 Benchmarking 15
7.6 Waste, Transport and Engagement 15
7.7 Scenario Analysis 16
**8. TESTING AND VALIDATION** **16**
8.1 Testing Strategy 16
8.2 Test Cases 16
8.3 Performance and Usability 17
**9. CHALLENGES AND FUTURE ENHANCEMENTS** **17**
9.1 Challenges and Mitigation 17
9.2 Limitations 18
9.3 Future Enhancements 18
**10. CONCLUSION** **18**
**REFERENCES** **19**
**APPENDIX: INSTALLATION AND EXECUTION** **19**

# LIST OF FIGURES


| | | |
|---|---|---|
| Figure 4.1 | Five-layer architecture of the system | 6 |
| Figure 4.2 | Data flow from device to dashboard | 7 |
| Figure 4.3 | Simplified entity-relationship diagram | 7 |
| Figure 6.1 | Management dashboard with KPIs, monthly scope trend and target reference lines | 11 |
| Figure 6.2 | Operations console: live building status, IoT health and campus map | 12 |
| Figure 7.1 | Emissions by source over the last 30 days | 13 |
| Figure 7.2 | Average working-day load profile by building type | 13 |
| Figure 7.3 | Grid draw and rooftop solar generation, Academic Block A | 13 |
| Figure 7.4 | After-hours air-conditioning load detected in Academic Block B | 14 |
| Figure 7.5 | Night-time minimum flow revealing a leak in Boys Hostel 1 | 14 |
| Figure 7.6 | Actual, fitted and forecast daily grid electricity | 15 |
| Figure 7.7 | Energy Performance Index by building | 15 |
| Figure 7.8 | Weekly waste diversion rate and commute mode share | 16 |


# LIST OF TABLES


| | | |
|---|---|---|
| Table 2.1 | Comparison of existing approaches with the proposed system | 3 |
| Table 3.1 | User roles and access rights | 3 |
| Table 3.2 | Functional requirements | 4 |
| Table 3.3 | Technology stack | 5 |
| Table 4.1 | Key REST API endpoints | 8 |
| Table 5.1 | Default emission factors (to be confirmed before disclosure) | 9 |
| Table 7.1 | Campus KPIs reported by the system (last 30 days) | 12 |
| Table 7.2 | Detection results for injected faults | 14 |
| Table 7.3 | Scenario: 50% LED retrofit, 250 kWp solar, +1 °C set-point, 30% occupancy controls, 2 e-buses | 16 |
| Table 8.1 | Automated test cases (18 of 18 passing) | 16 |
| Table 9.1 | Challenges and mitigation | 17 |


# LIST OF ABBREVIATIONS

| | |
|---|---|
| **API** | Application Programming Interface |
| **BEE** | Bureau of Energy Efficiency |
| **CEA** | Central Electricity Authority |
| **$CO_2e$** | Carbon dioxide equivalent |
| **EPI** | Energy Performance Index |
| **GHG** | Greenhouse Gas |
| **HVAC** | Heating, Ventilation and Air Conditioning |
| **IoT** | Internet of Things |
| **JWT** | JSON Web Token |
| **KPI** | Key Performance Indicator |
| **MAD** | Median Absolute Deviation |
| **MAPE** | Mean Absolute Percentage Error |
| **REST** | Representational State Transfer |
| **RBAC** | Role-Based Access Control |
| **STP** | Sewage Treatment Plant |
| **$tCO_2e$** | Tonnes of carbon dioxide equivalent |

# CHAPTER 1
# INTRODUCTION

## 1.1 Background

Higher-education campuses are significant contributors to local energy demand and emissions. A typical campus includes academic blocks, hostels, laboratories, libraries, canteens, sports facilities, staff housing and a vehicle fleet. In warm and humid regions such as Chennai, cooling loads make grid electricity the dominant source of emissions, and water is pumped, stored and treated on site at a further energy cost.

At the same time, campuses are well placed to lead. They have a captive population, control over their own infrastructure, research capability and an educational mission. A monitored, data-driven campus doubles as a living laboratory: the same data that helps the estate office find a leaking valve can be used by students for coursework in data science, energy systems and environmental studies.

The guiding principle of this project is simple: *what is measured gets managed*. Continuous monitoring turns sustainability from an annual compliance exercise into a daily operating practice.

## 1.2 Problem Statement

Current campus sustainability practice commonly suffers from the following gaps:

- Energy and water are recorded at the main meter only, so individual buildings and departments are invisible and cannot be held accountable.
- Carbon accounting is manual, periodic and spreadsheet-based, which makes it slow and error-prone.
- Faults such as leaks, idle air-conditioning and lights left on go unnoticed for days or weeks.
- Students and staff receive no feedback on the impact of their behaviour.
- Reports are hard to compare year on year because methods, factors and boundaries change.

## 1.3 Objectives

1. Monitor electricity, solar generation, water, indoor environment, waste and transport data in near real time.
2. Calculate campus emissions automatically using recognised, version-controlled emission factors aligned with the GHG Protocol.
3. Detect anomalies and wastage and route alerts to the responsible team as trackable work orders.
4. Provide role-specific dashboards for management, facility staff and the campus community.
5. Support target setting, demand forecasting, benchmarking and what-if scenario analysis.
6. Build a scalable, low-cost architecture using only free and open-source software that can start small and expand building by building.

## 1.4 Scope of the Project

The system covers measurement, data transmission, storage, analytics, visualisation, alerting and reporting. It addresses Scope 1 (generators, fleet, LPG, refrigerant leakage), Scope 2 (purchased electricity, location- and market-based) and a practical subset of Scope 3 (water supply and treatment, waste disposal and student and staff commuting). It does not replace existing building management systems; it integrates with them through a documented ingestion API. Because physical meters were not available during development, a physics-based simulator generates realistic sensor data so that

every layer of the system can be exercised end to end.

### 1.5 Organisation of the Report

Chapter 2 reviews the standards and existing approaches that informed the design. Chapter 3 lists the functional and non-functional requirements. Chapter 4 presents the system architecture, data flow and database design, and Chapter 5 explains the carbon accounting method and the analytical algorithms. Chapter 6 describes the implementation, Chapter 7 discusses the results, Chapter 8 covers testing, and Chapters 9 and 10 present limitations, future enhancements and conclusions.

## CHAPTER 2
# LITERATURE SURVEY AND STANDARDS

### 2.1 Carbon Accounting Standards

The **GHG Protocol Corporate Accounting and Reporting Standard** is the most widely used framework for organisational inventories. It classifies emissions into three scopes and requires the organisation to define its boundary (operational or financial control) and a base year against which progress is measured. Its Scope 2 guidance requires dual reporting: a *location-based* figure using the average grid factor and a *market-based* figure that reflects contractual purchases of renewable power.

**ISO 14064-1** specifies principles for quantifying and reporting GHG emissions and removals, while **ISO 50001** defines energy-management systems built around Energy Performance Indicators (EnPIs) and baselines. For India, the **Central Electricity Authority (CEA) $CO_2$ Baseline Database** publishes the grid emission factor, recently in the region of 0.7-0.8 kg $CO_2$/kWh; because the value is revised periodically, a monitoring system must store factors as dated, versioned parameters.

### 2.2 Campus Sustainability Rating Systems

Green-building rating systems such as **GRIHA** and **IGBC** assess energy performance (kWh/$m^2$/year), water efficiency, waste management and renewable energy. **AASHE STARS**, used by universities worldwide, adds operations, engagement and curriculum credits, and NIRF-style disclosures in India increasingly ask for sustainability data. All of them reward measured, verifiable data over estimates.

The UN Sustainable Development Goals most relevant to this work are SDG 7 (affordable and clean energy), SDG 11 (sustainable cities), SDG 12 (responsible consumption) and SDG 13 (climate action).

### 2.3 Existing Approaches

*Table 2.1: Comparison of existing approaches with the proposed system*

| Approach | Strengths | Limitations |
|---|---|---|
| Annual utility-bill audit | Cheap, credible totals | No building detail; results arrive months late |
| Building Management System (BMS) | Real-time control of HVAC | Vendor-specific; no carbon accounting or community view; expensive licences |
| Commercial energy dashboards | Polished visualisation | Subscription cost; electricity only; data held by vendor |
| Spreadsheet carbon calculators | Flexible, familiar | Manual, error-prone, no alerts, poor versioning |
| **Proposed system** | Integrated multi-resource monitoring, GHG engine, alerts, forecasting and engagement; free software | Requires metering investment for real deployment |

## 2.4 Analytical Techniques

Research on building energy analytics shows that simple, explainable models perform well for campus loads, which are driven mainly by occupancy schedules and weather. Robust statistics such as the median and median absolute deviation (MAD) are widely recommended for anomaly detection because, unlike the mean and standard deviation, they are not distorted by the very anomalies being sought. Night-time minimum flow analysis is the standard method used by water utilities to detect leaks, and multiple linear regression with calendar and temperature terms is a common baseline for short-term demand forecasting. These findings guided the choice of transparent algorithms that can be explained, audited and run on modest hardware.

# CHAPTER 3
# REQUIREMENTS ANALYSIS

## 3.1 Stakeholders and Roles

*Table 3.1: User roles and access rights*

| Role | Typical users | Capabilities |
|---|---|---|
| Administrator | Sustainability officer, management | All dashboards; campus settings and boundary; emission factors; targets; buildings, meters and gateways; user roles |
| Facility team | Estate office, electricians, plumbers | Operations console; acknowledge and resolve alerts; run anomaly scans; log fuel, refrigerant and waste |
| Community | Students and staff | Read-only dashboards; hostel and department leaderboards; private commute footprint calculator |

## 3.2 Functional Requirements

*Table 3.2: Functional requirements*

| ID | Requirement |
|---|---|
| FR1 | Ingest meter readings from authenticated gateways; reject malformed data and flag out-of-range values. |
| FR2 | Convert activity data to kg $CO_2$e by scope, source, building and period using the factor valid on each day. |
| FR3 | Show campus KPIs: total and per-student emissions, kWh/$m^2$, renewable share, water per person, waste diversion and progress to target. |
| FR4 | Detect after-hours load, spikes, leaks, stuck or offline meters and high $CO_2$, and create de-duplicated alerts with estimated cost. |
| FR5 | Allow alerts to be acknowledged, annotated, assigned and resolved (work-order tracking). |
| FR6 | Forecast daily electricity demand, emissions and cost with a prediction interval. |
| FR7 | Benchmark buildings and model the savings, cost and payback of interventions. |
| FR8 | Record waste weighings, fuel and refrigerant use, trees and commute surveys. |
| FR9 | Generate a GHG inventory PDF and CSV exports for any date range. |
| FR10 | Register users, authenticate with tokens and enforce role-based permissions. |

## 3.3 Non-Functional Requirements

- **Cost:** every component must be free and open source; no paid APIs or cloud subscriptions.
- **Security:** hashed passwords, JWT access tokens with refresh rotation, per-gateway device keys, role-based authorisation and rate limiting on public endpoints.
- **Data integrity:** suspicious values are flagged, never silently corrected or deleted; factor history is append-only so past results remain reproducible.
- **Privacy:** commute answers are visible only to the person who submitted them; only aggregated totals are reported.
- **Performance:** dashboard queries should return in under one second for a year of hourly data on a laptop.
- **Usability and portability:** responsive interface for desktop and mobile; runs on Windows, Linux or macOS with a one-step setup.

## 3.4 Hardware and Software Requirements

*Table 3.3: Technology stack*

| Layer | Choice (all free) | Purpose |
|---|---|---|
| Backend language | Python 3.12 | Services, analytics and management commands |
| Web framework | Django 5.2, Django REST Framework 3.17 | REST API, ORM, admin site |
| Authentication | djangorestframework-simplejwt | JSON Web Tokens with refresh rotation |
| Database | SQLite (PostgreSQL/TimescaleDB ready) | Readings, metadata, factors and logs |
| Numerics and reports | NumPy, ReportLab | Regression forecasting; PDF generation |
| Frontend | React 19, TypeScript, Vite | Single-page application |
| UI libraries | TanStack Query, Zustand, Recharts, Leaflet + OpenStreetMap | Data fetching, state, charts, campus map |
| Hardware (deployment) | ESP32 boards, Modbus meters, ultrasonic flow meters | Low-cost sensing |
| Development machine | Any PC with 4 GB RAM | Runs all services locally |

# CHAPTER 4
# SYSTEM DESIGN

## 4.1 Layered Architecture

The system follows the five-layer architecture proposed in the project report. Each layer has a single responsibility and communicates with its neighbours through well-defined interfaces, so that, for example, a simulated meter can be replaced by a real Modbus meter without any change above the connectivity layer.

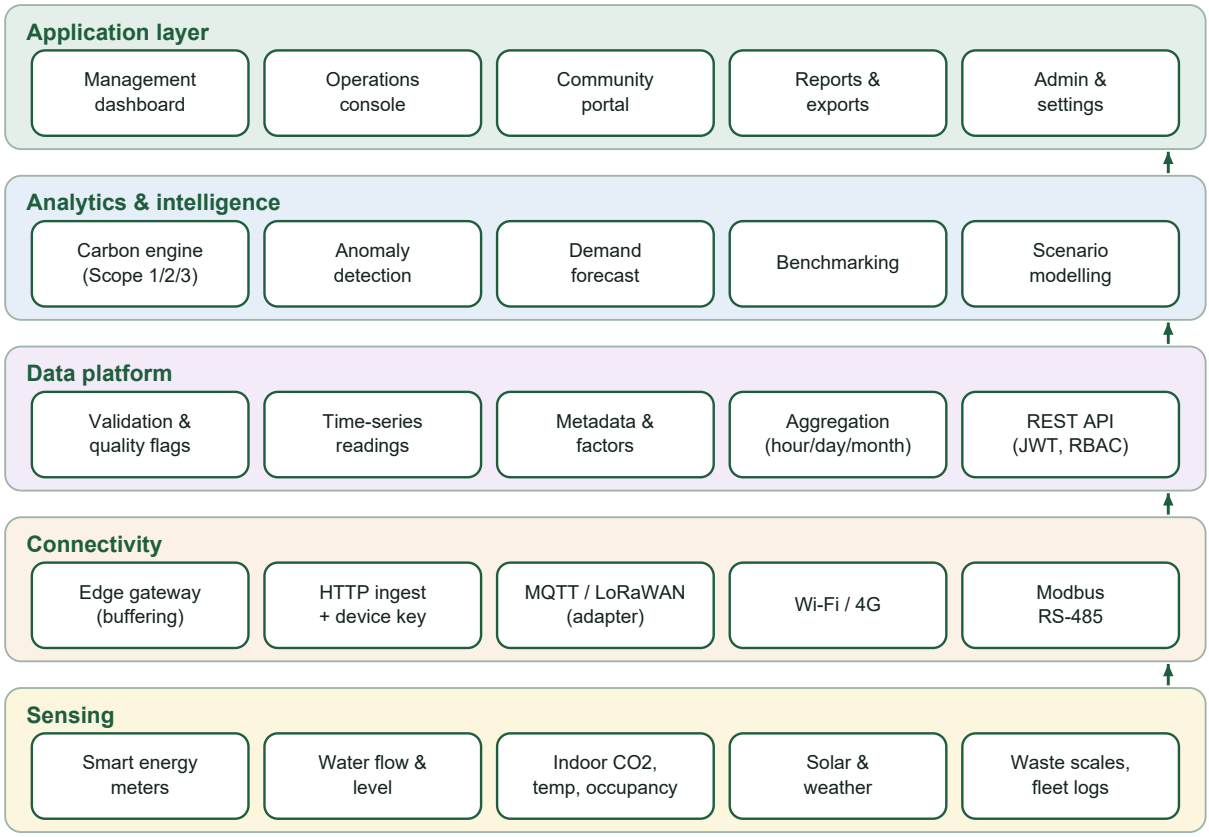



*Figure 4.1: Five-layer architecture of the system*

- **Sensing:** energy meters at each building incomer, rooftop solar meters, water meters per building and per source, indoor environment sensors, a weather station, waste scales and fleet logs.
- **Connectivity:** edge gateways buffer readings during outages and forward them over HTTP with a secret key; MQTT or LoRaWAN can be bridged through the same adapter.
- **Data platform:** validation, quality flags, time-series storage and aggregation to hourly, daily and monthly values; metadata such as buildings, tariffs and factors in relational tables.
- **Analytics:** carbon engine, anomaly detection, forecasting, benchmarking and scenario modelling.
- **Application:** management dashboard, operations console, community portal and reporting.

## 4.2 Data Flow

Figure 4.2 shows how a reading travels through the system. Devices push readings to the gateway, which authenticates with the ingestion endpoint. Valid readings are stored; out-of-range readings are stored with a *suspect* flag and excluded from totals. Analytics services read the store on demand when a dashboard calls the REST API, and a periodic scan writes alerts that appear in the operations console.

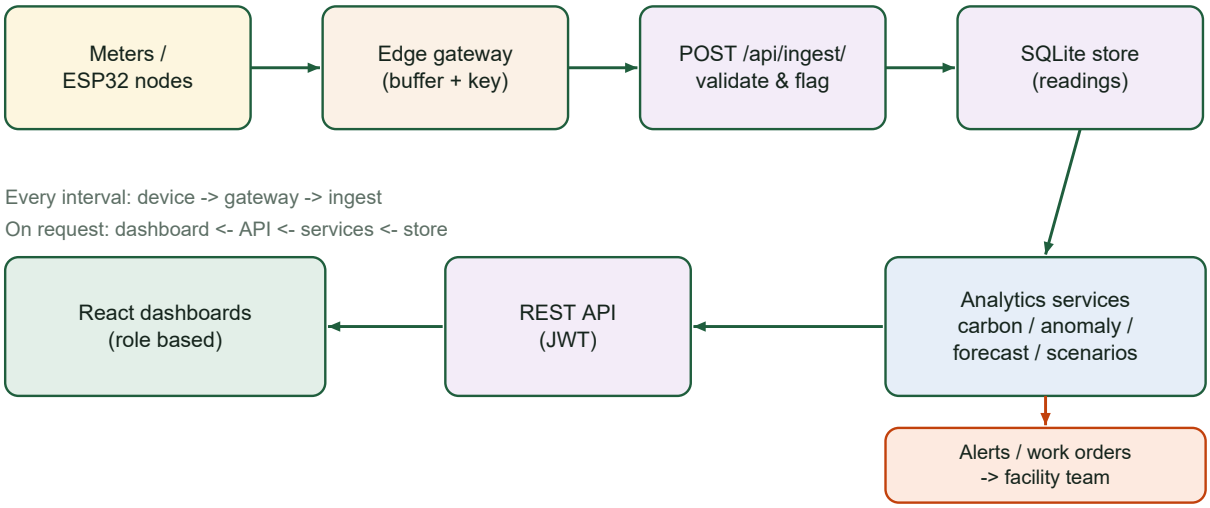



*Figure 4.2: Data flow from device to dashboard*

## 4.3 Database Design

The relational schema separates the physical campus (buildings, gateways, meters), measurements (readings, indoor environment, weather), logged activity data (fuel, waste, commuting, trees), configuration (emission factors, targets, calendar, campus settings) and operations (alerts, users). Readings are indexed on (meter, timestamp) so that hourly and daily aggregation stays fast.

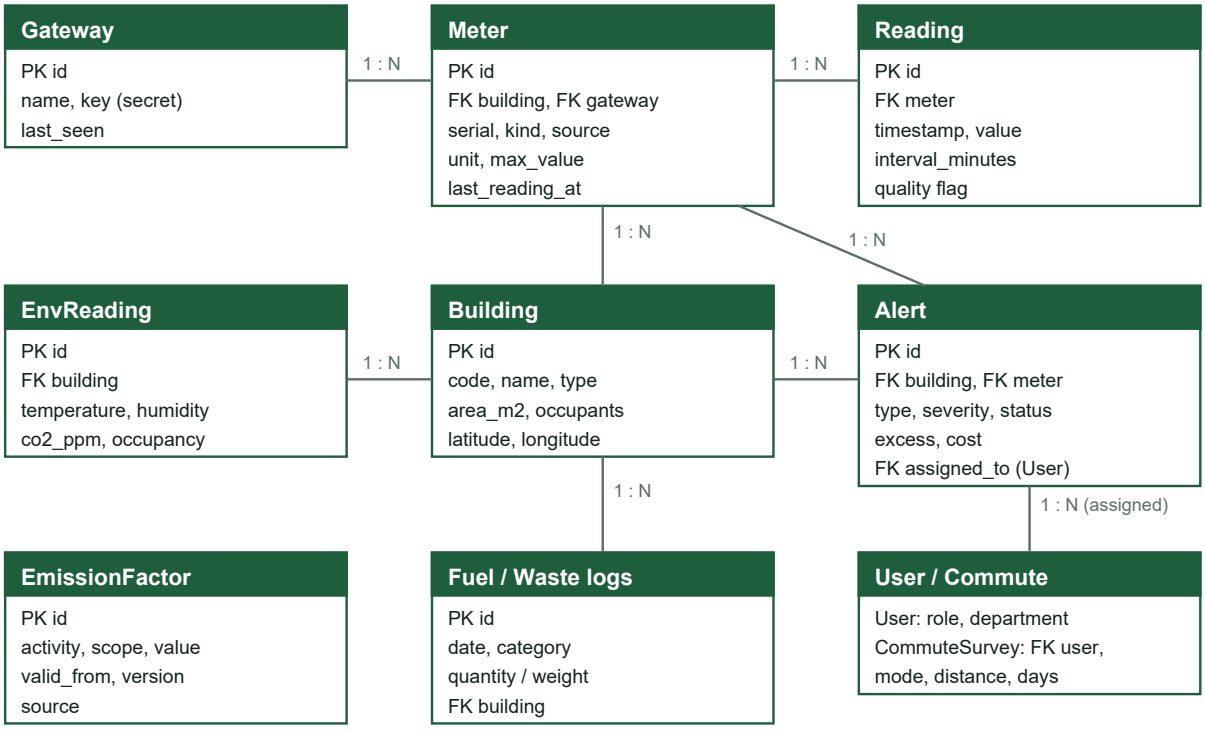



*Figure 4.3: Simplified entity-relationship diagram*

### 4.4 API Design

All functionality is exposed through a versionable REST API under `/api/`. The main endpoints are listed below; further CRUD endpoints exist for buildings, meters, gateways, factors, logs, trees, targets and users.

*Table 4.1: Key REST API endpoints*

| Endpoint | Method | Description |
|---|---|---|
| /api/auth/login/, /refresh/, /register/ | POST | Obtain and rotate JWT tokens; self-registration |
| /api/ingest/ | POST | Gateway ingestion (X-Gateway-Key header) |
| /api/analytics/overview/ | GET | Campus KPIs, scope split, monthly trend, target progress |
| /api/analytics/live/ | GET | Current demand, solar, water flow and indoor status per building |
| /api/analytics/forecast/ | GET | Demand forecast with 95% interval and model metrics |
| /api/analytics/scenarios/ | POST | Savings, cost and payback for chosen interventions |
| /api/alerts/{id}/ | PATCH | Acknowledge, assign, annotate or resolve an alert |
| /api/reports/ghg.pdf | GET | GHG inventory report for a date range |

### 4.5 Security Design

Passwords are stored with Django's PBKDF2 hasher. Users authenticate once and receive a short-lived access token (30 minutes) and a refresh token (7 days) that is rotated on use. Every API view declares a permission class: read-only for all signed-in users, write access restricted to administrators or facility staff as appropriate. Devices never use user accounts; each gateway has its own random 160-bit key that can be revoked independently. Anonymous endpoints are rate-limited, and administrators cannot remove their own privileges by mistake.

## CHAPTER 5
# METHODOLOGY AND ALGORITHMS

### 5.1 Carbon Accounting Method

Every emission line is calculated with the basic relationship of the GHG Protocol:

$$E_d\,(kg\ CO_2e) = A_d \times EF_d$$

where $A_d$ is the activity data on day $d$ (kWh, litres, kg, kL or passenger-km) and $EF_d$ is the emission factor whose *valid_from* date is the latest on or before $d$. Applying factors day by day means that publishing a new CEA value changes only the days after it takes effect, and every historical report can be reproduced exactly. Market-based Scope 2 is computed as the location-based value multiplied by (1 - certified green power share).

Commuting is estimated from anonymous surveys. For respondents $i$ using mode $m$, the daily passenger-km scaled to the whole population P from n responses is:

$$km_m = (P / n) \times (44 / 52) \times \Sigma_{i\ in\ m}\ 2 \cdot dist_i \cdot days_i / 7$$

where 44/52 spreads the academic year over the calendar. Tree sequestration is calculated separately and never subtracted from emissions, following the conservative approach recommended in the

project report.

*Table 5.1: Default emission factors (to be confirmed before disclosure)*

| Activity | Scope | Factor | Source (indicative) |
|---|---|---|---|
| Grid electricity | 2 | 0.716 kg/kWh (v2, from Apr 2025) | CEA $CO_2$ Baseline Database |
| Diesel (generators, buses) | 1 | 2.68 kg/L | IPCC 2006 / DEFRA |
| LPG (canteen, labs) | 1 | 2.98 kg/kg | IPCC 2006 |
| Refrigerant R-410A / R-32 | 1 | 2088 / 675 kg/kg | IPCC AR4 GWP-100 |
| Water supply and treatment | 3 | 0.344 kg/kL | DEFRA (indicative) |
| Waste to landfill / recycled / composted | 3 | 0.497 / 0.021 / 0.009 kg/kg | DEFRA (indicative) |
| Commute: car / two-wheeler / bus / metro | 3 | 0.171 / 0.083 / 0.105 / 0.035 kg/km | DEFRA, indicative |

## 5.2 Anomaly Detection

For every electricity meter, the normal profile is learnt separately for each hour of the day and for working and non-working days (weekends, holidays and vacations from the academic calendar) over the previous 28 days. For a recent hourly value x with historical values H in the same bucket, a robust z-score is computed:

$$z = (x - \text{median}(H)) / \max(1.4826 \cdot \text{MAD}(H),\ 0.05 \cdot \text{median}(H),\ 1)$$

An hour is flagged when $z > 4$ and the excess exceeds both 20% of the median and 25 kWh, which suppresses alarms on tiny loads. Flagged hours are grouped by building and operational date (hours after midnight belong to the previous evening). Groups between 22:00 and 06:00 in non-residential buildings become *after-hours* alerts; others become *spike* alerts. Severity is graded on the excess energy, and the estimated cost is excess × tariff.

**Leak detection** uses the minimum hourly flow between 01:00 and 05:00, when legitimate use is close to zero. If the minimum on a recent night exceeds max(2.5 × baseline, baseline + 0.5 kL/h), where the baseline is the median of the previous four weeks' night minima, one continuing leak work order is opened for that meter.

**Data-quality checks** raise alerts for six identical non-zero readings (stuck sensor), meters silent for more than three hours (offline) and readings above the meter's physical limit (impossible value). **Indoor environment** alerts are raised when $CO_2$ exceeds 1200 ppm. Every alert carries a unique de-duplication key, so repeated scans never create duplicates.

## 5.3 Demand Forecasting

Daily grid electricity $y_t$ is modelled by multiple linear regression fitted with NumPy least squares on the last 120 days:

$$y_t = \beta_0 + \beta_1 \cdot t + \Sigma_{k=1..6}\, \gamma_k \cdot DOW_k + \delta \cdot Holiday_t + \theta \cdot (T_t - 28) + \varepsilon_t$$

where DOW are day-of-week indicators (Sunday as reference), Holiday marks weekday holidays and vacations, and T is the mean outdoor temperature. The coefficient θ directly gives the cooling sensitivity in kWh per degree. Future days use the calendar and the recent average temperature; the 95% prediction interval is ±1.96σ of the residuals. Accuracy is reported as MAPE and $R^2$.

### 5.4 Benchmarking and Scenario Modelling

Buildings are compared on the Energy Performance Index, EPI = annualised kWh / floor area ($m^2$), together with kWh and litres of water per occupant per day. Star bands are modelled on BEE building rating ranges.

The scenario model starts from annualised grid consumption E (from the last 30 days) and applies transparent engineering assumptions that the user can change: LED retrofit saves 50% of the lighting share (20% of E) for the retrofitted fraction; rooftop solar yields 1400 kWh per kWp per year; each 1 °C increase in AC set-point saves 6% of the cooling share (45% of E); occupancy controls save 15% of cooling and lighting in covered areas; an electric bus replaces diesel at 4 km/L but adds 1.1 kWh/km of charging. For each measure the model reports kWh and diesel saved, $tCO_2e$ avoided, annual saving, capital cost and simple payback = capex / annual saving.

## CHAPTER 6
# IMPLEMENTATION

### 6.1 Project Structure

The backend is a Django project with four applications. **accounts** holds the custom user model with roles, registration and permission classes. **monitoring** models buildings, gateways, meters, readings, indoor and weather data, the academic calendar and alerts, and implements the ingestion endpoint. **sustainability** stores emission factors, fuel and refrigerant logs, waste weighings, commute surveys, trees and targets. **analytics** contains the services described in Chapter 5, the REST views, and two management commands: `seed_demo`, which builds a realistic campus with 120 days of hourly history, and `simulate`, which acts as a live gateway. The frontend is a React and TypeScript single-page application organised into pages, shared components, an API client and small state stores.

### 6.2 Carbon Engine

The `FactorBook` class loads every factor version once and answers which value applied on a given day; the engine then produces one line item per building, day and activity. All dashboards, the PDF report and the CSV export are computed from the same line items, which guarantees that the numbers agree everywhere.

```
class FactorBook:
    """Loads every factor version once; answers 'which value applied on this day?'."""
    def __init__(self):
        self._rows = defaultdict(list)
        for factor in EmissionFactor.objects.order_by("valid_from"):
            self._rows[factor.activity].append(factor)

    def get(self, activity, on=None):
        rows = self._rows.get(activity)
        if not rows:
            return None
        on = on or timezone.localdate()
        chosen = rows[0]
        for factor in rows:
            if factor.valid_from <= on:
                chosen = factor
            else:
                break
        return chosen
```

## 6.3 IoT Ingestion and Simulation

Gateways post batches of up to 5000 readings to `/api/ingest/`. Each reading carries the meter serial, the consumption during the interval and optionally the interval length and timestamp. The endpoint authenticates the gateway key, rejects unknown meters, non-numeric or future values with a per-item error, flags values above the meter's hourly limit as suspect, stores the rest in one bulk insert and updates each meter's last-seen time.

```
POST /api/ingest/        X-Gateway-Key: 6e73...bc69
{"readings": [{"serial": "EM-ACA", "value": 42.5, "interval_minutes": 15},
              {"serial": "WM-BH1", "value": 0.61, "interval_minutes": 15}]}
-> 201 {"accepted": 2, "rejected": []}
```

Because physical meters were not available, the simulator models each building from first principles: an always-on base load, an occupancy-driven load that follows the timetable of its building type, and a cooling load proportional to the outdoor temperature above 24 °C. Solar output follows irradiance with day-to-day cloud cover, water use follows daily routines, and the weather is Chennai-like with monsoon rain in October. Day-level randomness is seeded by date, so historical and live data agree. The seed command deliberately injects six faults (air-conditioning left on for two nights, a hostel tank leak, a lab test rig left on, a poorly ventilated hall and an impossible meter value) to validate detection.

## 6.4 User Interface

The web application provides thirteen screens grouped into Overview, Modules, Intelligence and Administration. Server state is cached with TanStack Query; the live console refreshes every 30 seconds. Charts use Recharts and the campus map uses Leaflet with free OpenStreetMap tiles. The layout adapts to mobile screens, and pages are code-split so that the initial download stays small.

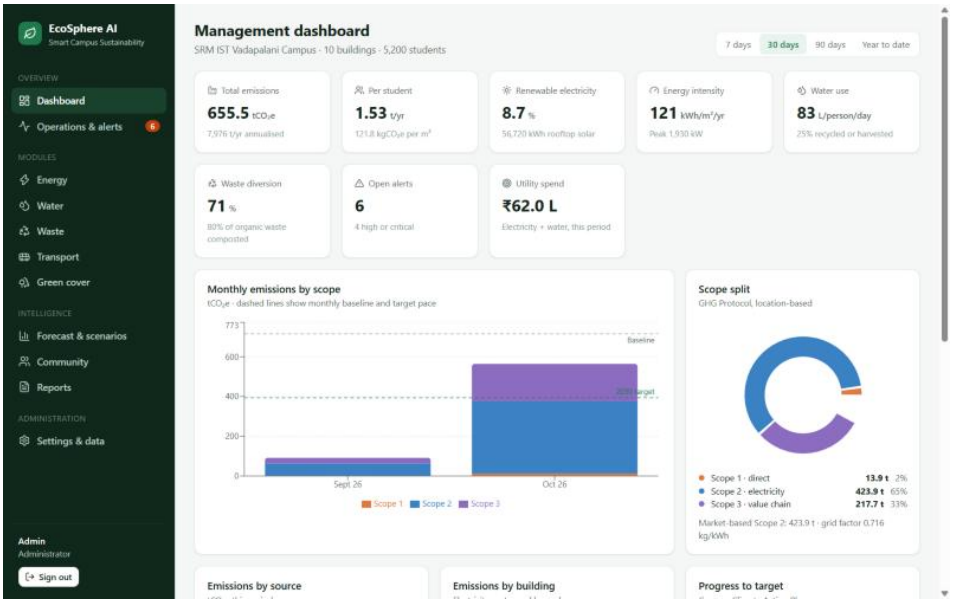



*Figure 6.1: Management dashboard with KPIs, monthly scope trend and target reference lines*

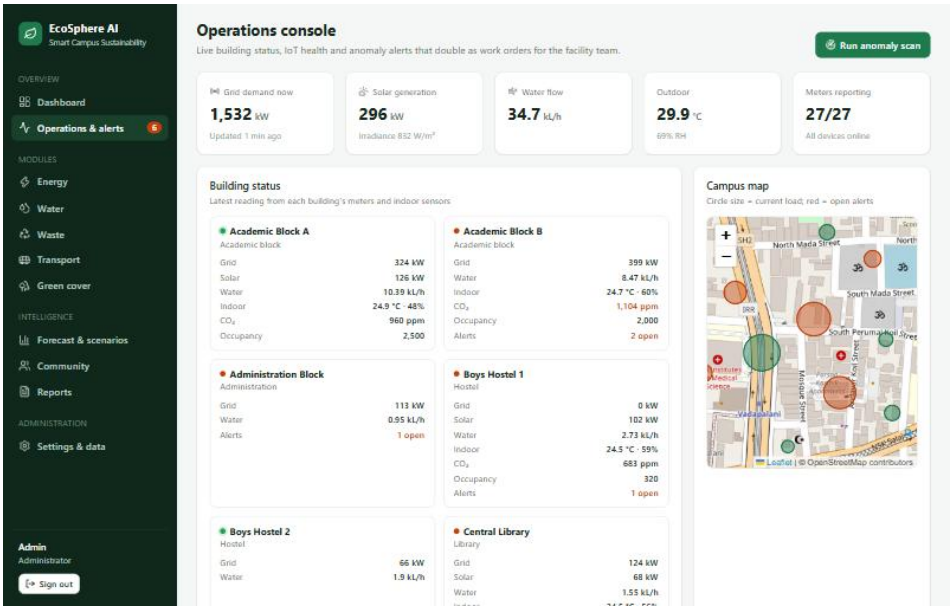



*Figure 6.2: Operations console: live building status, IoT health and campus map*

# CHAPTER 7
# RESULTS AND DISCUSSION

## 7.1 Campus Footprint

Table 7.1 summarises the key performance indicators reported by the system for the most recent 30 days of the evaluation dataset (ten buildings, 65,500 $m^2$, 5,200 students and 650 staff).

*Table 7.1: Campus KPIs reported by the system (last 30 days)*

| Indicator | Value | Indicator | Value |
|---|---|---|---|
| Total emissions (30 days) | 655.5 $tCO_2e$ | Renewable share of electricity | 8.7% |
| Annualised emissions | 7,975 $tCO_2e$/yr | Energy Performance Index | 120.5 kWh/$m^2$/yr |
| Per student | 1.53 t/yr | Peak demand | 1,930 kW |
| Per floor area | 121.8 kg/$m^2$/yr | Water use | 82.5 L/person/day |
| Scope 1 / 2 / 3 | 13.9 / 423.9 / 217.7 t | Recycled + harvested water | 25.0% |
| Electricity cost (30 days) | Rs. 53.3 lakh | Waste diversion rate | 71.3% |
| Tree sequestration | 20.2 $tCO_2$/yr | Sustainable commute share | 64.0% |

Purchased electricity dominates the footprint at about 65% of the total, as expected for a cooling-dominated campus in South India, followed by student and staff commuting. Direct Scope 1 sources are small, but refrigerant leakage stands out: a single 6 kg R-410A top-up equals about 12.5 $tCO_2e$, more than a month of the entire bus fleet. Tree sequestration offsets only about 0.25% of annual emissions, which confirms that planting is not a substitute for reducing consumption.

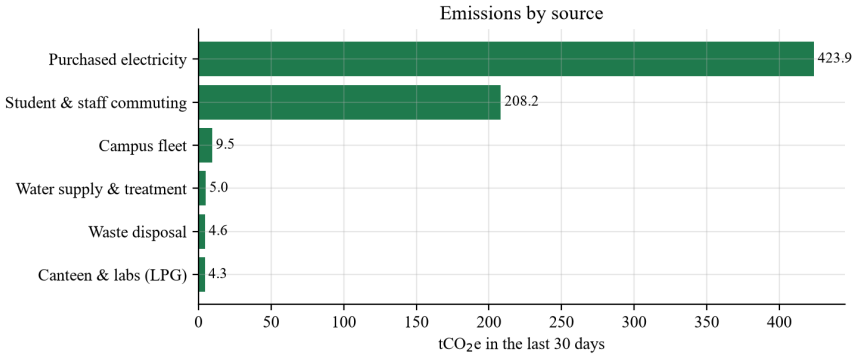



*Figure 7.1: Emissions by source over the last 30 days*

## 7.2 Energy Use Patterns

Building-level metering reveals patterns that a single main meter hides. Academic blocks and laboratories follow the timetable, with laboratories retaining a high base load from equipment; hostels peak in the evening; the food court shows breakfast, lunch and dinner peaks. Rooftop solar on Academic Block A visibly shaves the midday grid draw.

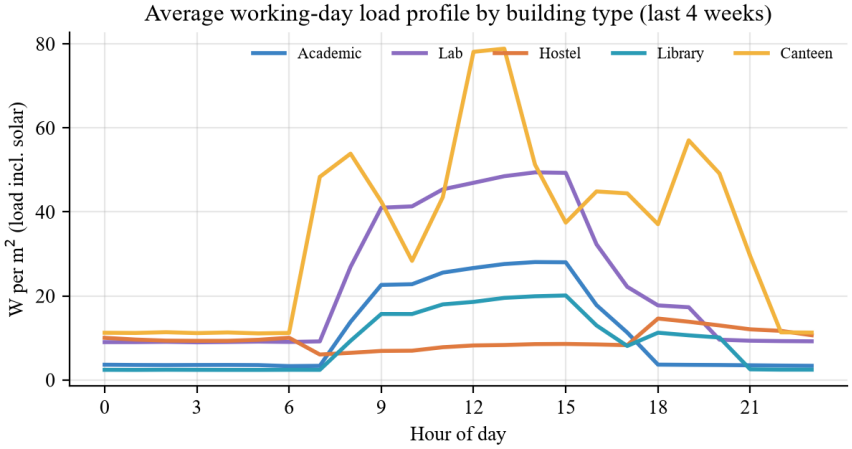



*Figure 7.2: Average working-day load profile by building type*

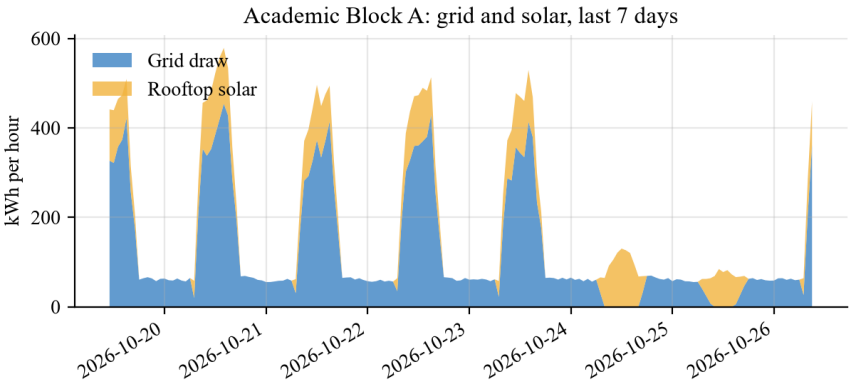



*Figure 7.3: Grid draw and rooftop solar generation, Academic Block A*

## 7.3 Fault Detection

The anomaly scan raised exactly six alerts on the evaluation dataset, one for each injected fault, with no false positives on the 120 days of normal data. Figure 7.4 shows the after-hours alert for Academic Block B: night-time load rose from about 50 kWh to over 200 kWh per hour for two nights while the learnt profile stayed flat, an excess of roughly 1,290 kWh (about Rs. 11,600) per night. Figure 7.5 shows the leak signature in Boys Hostel 1, where minimum night flow jumped from about 0.2 to 2.6 kL/h, an estimated loss of 58 kL per day. In a manual, bill-based process such faults would typically surface only weeks later.

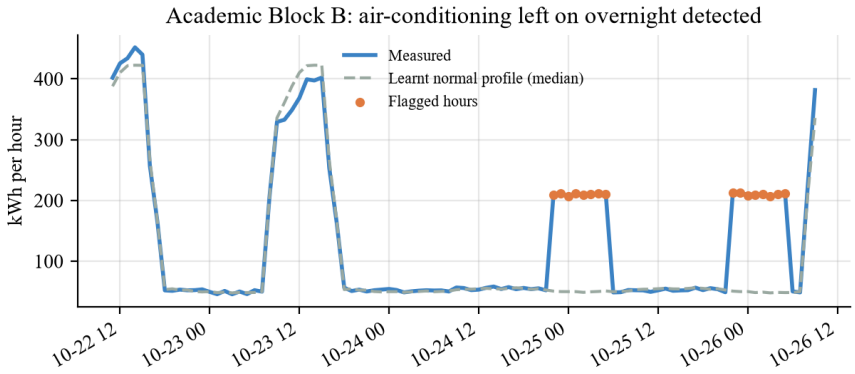



*Figure 7.4: After-hours air-conditioning load detected in Academic Block B*

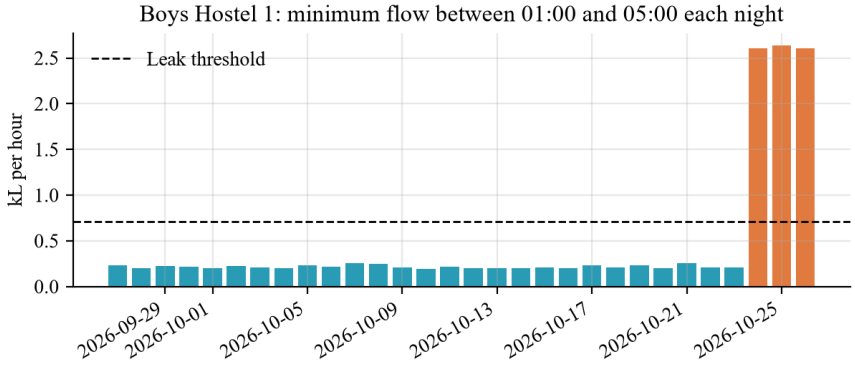



*Figure 7.5: Night-time minimum flow revealing a leak in Boys Hostel 1*

*Table 7.2: Detection results for injected faults*

| Injected fault | Detected as | Severity | Estimated impact |
|---|---|---|---|
| AC left on overnight (two nights), Academic Block B | After-hours load x2 | High | ~1,290 kWh per night |
| Leaking tank float valve, Boys Hostel 1 | Possible water leak | High | ~58 kL per day |
| Test rig left at full power, Tech Park Labs | Electricity spike | High | ~780 kWh |
| Packed hall with fresh air off, Central Library | Poor ventilation | Medium | $CO_2$ ~1,490 ppm |
| 99,999 kWh reading, Administration Block | Impossible value | Low | Flagged, excluded |

## 7.4 Forecasting Accuracy

The regression model was trained on 120 days and achieved a mean absolute percentage error of 2.36% and $R^2$ of 0.987. It captures the weekly cycle, the lower demand during holidays and vacations, and a cooling sensitivity of about 935 kWh per day for each additional degree Celsius. The 30-day forecast predicts 614 MWh, 439 $tCO_2e$ and a bill of about Rs. 55.2 lakh, which supports budgeting and peak-load planning.

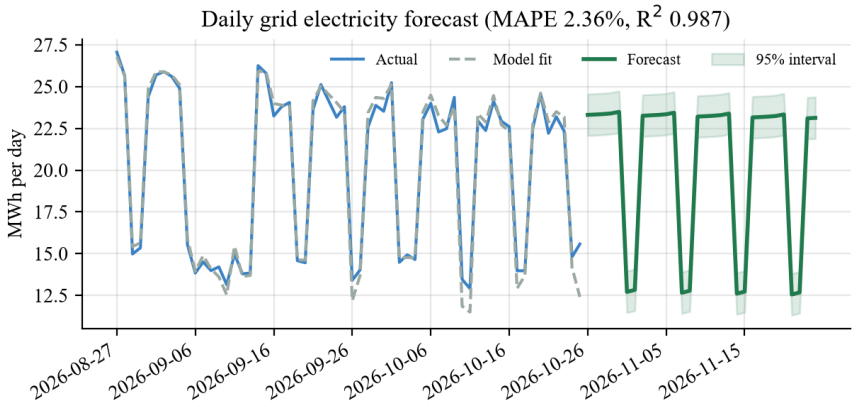



*Figure 7.6: Actual, fitted and forecast daily grid electricity*

## 7.5 Benchmarking

The food court has the highest Energy Performance Index (255 kWh/$m^2$/yr) because of cooking and refrigeration in a small floor area, followed by Tech Park Labs (157). The sports complex is the most efficient. Normalising by area and occupancy makes such comparisons fair and points retrofit budgets at the right buildings.

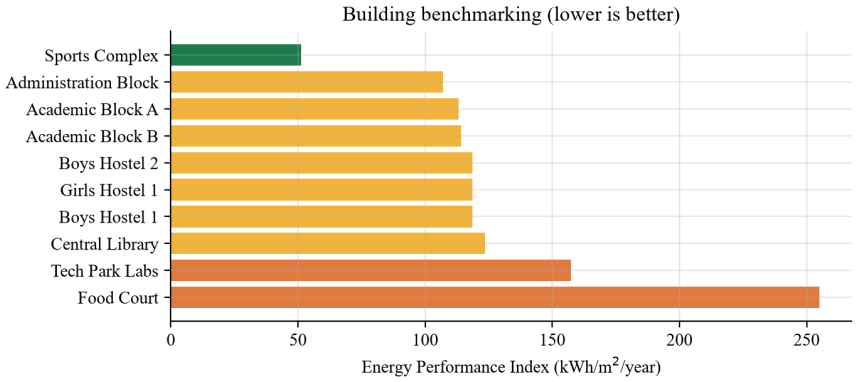



*Figure 7.7: Energy Performance Index by building*

## 7.6 Waste, Transport and Engagement

The waste module shows a diversion rate of 70.1%, with 80% of organic waste composted or converted to biogas, and the weekly trend in Figure 7.8 reflects the improvement from a segregation drive. Diverting waste from landfill avoided about 42.02 $tCO_2e$ over the period. Commute surveys show 64% of trips by walking, cycling, bus, metro or electric two-wheeler, yet commuting still contributes about 2,533 $tCO_2e$ per year, mostly from cars and petrol two-wheelers. The hostel leaderboard converts week-on-week reductions into points, giving residents visible feedback.

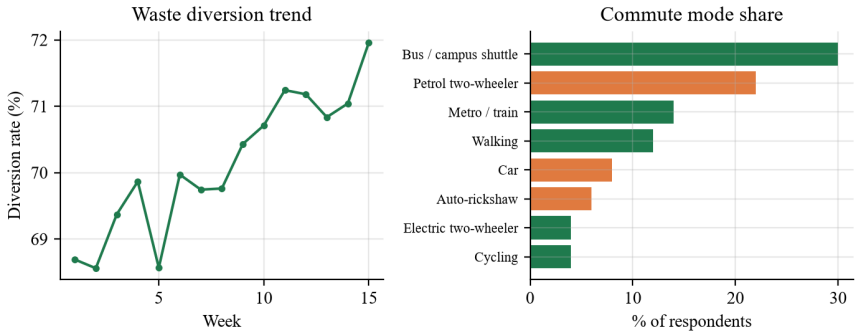



*Figure 7.8: Weekly waste diversion rate and commute mode share*

## 7.7 Scenario Analysis

*Table 7.3: Scenario: 50% LED retrofit, 250 kWp solar, +1 °C set-point, 30% occupancy controls, 2 e-buses*

| Measure | $tCO_2e$/yr | Saving/yr | Capex | Payback |
|---|---|---|---|---|
| LED lighting retrofit | 260 | Rs. 32.7 L | Rs. 39.3 L | 1.2 yr |
| Rooftop solar expansion | 251 | Rs. 31.5 L | Rs. 112.5 L | 3.6 yr |
| Air-conditioning set-point increase | 140 | Rs. 17.7 L | Nil | Immediate |
| Occupancy-based HVAC & lighting controls | 152 | Rs. 19.1 L | Rs. 19.6 L | 1.0 yr |
| Electric campus buses | -6 | Rs. 6.8 L | Rs. 160.0 L | 23.5 yr |
| **Total** | **797** | **Rs. 107.8 L** | **Rs. 331.4 L** | **3.1 yr** |

The combined package would cut grid electricity by 14.7% and avoid about 797 $tCO_2e$ per year with a payback of 3.1 years. Raising air-conditioning set-points costs nothing, and LED retrofits and occupancy controls pay back in about a year. The model also exposes a counter-intuitive result: on today's coal-heavy grid, electric buses save money but slightly *increase* emissions unless they are charged from solar, an insight that would be easy to miss without quantitative modelling.

# CHAPTER 8
# TESTING AND VALIDATION

## 8.1 Testing Strategy

Testing combined automated end-to-end tests, manual interface testing for every role and screen, and data validation against known inputs. The automated suite seeds a fresh 45-day campus in an isolated test database and exercises the carbon engine, anomaly detection, ingestion, permissions and every analytics endpoint. The frontend is type-checked by the TypeScript compiler on every build.

## 8.2 Test Cases

*Table 8.1: Automated test cases (18 of 18 passing)*

| # | Test case | Expected result | Status |
|---|---|---|---|
| 1 | Electricity line item equals kWh × 0.716 | Exact match for every building-day | Pass |
| 2 | New factor version from a given date | Old value before, new value after | Pass |
| 3 | Suspect 99,999 kWh reading | Excluded from totals | Pass |

| # | Test case | Expected result | Status |
|---|---|---|---|
| 4 | All three scopes present in 30-day inventory | Scopes 1, 2 and 3 reported | Pass |
| 5 | Injected faults | After-hours, spike, leak, $CO_2$ and impossible-value alerts | Pass |
| 6 | Repeated anomaly scan | No duplicate alerts | Pass |
| 7 | Ingest batch with bad items | 2 accepted, 2 rejected with reasons, 1 flagged | Pass |
| 8 | Ingest with wrong gateway key | HTTP 401 | Pass |
| 9 | Login and profile | Token issued; role returned | Pass |
| 10 | Anonymous access to analytics | HTTP 401 | Pass |
| 11 | Student writes factors, logs, alerts, users | HTTP 403 for each | Pass |
| 12 | Facility resolves an alert | Status resolved, timestamp set | Pass |
| 13 | Administrator demotes self | HTTP 400, refused | Pass |
| 14 | Commute answers privacy | User sees only own answers | Pass |
| 15 | All 16 analytics/report endpoints | HTTP 200 | Pass |
| 16 | Invalid date ranges and datasets | HTTP 400 with message | Pass |
| 17 | 14-day forecast | 14 points, MAPE below 10% | Pass |
| 18 | GHG report download | Valid PDF document | Pass |

### 8.3 Performance and Usability

With about 78,000 hourly readings, the campus overview endpoint responds in about 0.2 s, a year-to-date overview in under 1 s and the GHG PDF in about 1 s on an ordinary laptop. Seeding 120 days of data takes about seven seconds. Manual tests confirmed that each role sees only the navigation and actions it is allowed, that the live console updates as the simulator streams readings, that forms validate input and show clear error messages, and that the layout works without horizontal scrolling on a 375-pixel mobile screen.

## CHAPTER 9
# CHALLENGES AND FUTURE ENHANCEMENTS

### 9.1 Challenges and Mitigation

*Table 9.1: Challenges and mitigation*

| Challenge | Mitigation adopted |
|---|---|
| High upfront cost of metering | Free software stack; phased, building-by-building rollout; ESP32-class low-cost sensing where accuracy needs are modest |
| Legacy and incompatible meters | Gateway adapter pattern; simple HTTP ingestion that any device can call |
| Poor data quality | Validation, suspect flags, stuck/offline checks, completeness KPIs, cross-checks with utility bills |
| Changing emission factors | Dated, versioned factor register; append-only history |
| Privacy of personal travel data | Anonymous aggregation; individuals see only their own answers |
| Low community engagement | Leaderboards, points, personal footprint and practical tips |

## 9.2 Limitations

- The evaluation uses simulated data; accuracy must be re-validated against real meters and bills during a pilot.
- Default emission factors are indicative and must be replaced with the latest official values.
- SQLite suits a pilot; a campus-wide deployment with minute-level data should move to PostgreSQL with TimescaleDB, which the ORM supports without code changes.
- Scope 3 excludes purchased goods and business travel.

## 9.3 Future Enhancements

- AI-assisted control that adjusts air-conditioning set-points and lighting automatically from occupancy and weather.
- A digital twin of the campus for testing interventions virtually.
- Native MQTT and LoRaWAN adapters and WebSocket push for sub-minute live views.
- Integration with an internal carbon price or green fund that reinvests verified savings.
- Tamper-evident (hash-chained) logs for auditable carbon records.
- Smart-grid features such as demand response and battery storage coordination.
- A shared regional platform so that neighbouring institutions can benchmark against each other.

# CHAPTER 10
# CONCLUSION

This project delivered a complete, working Smart Campus Sustainability and Carbon Footprint Monitoring System that gives an institution what annual reports cannot: timely, granular and trustworthy information about its environmental impact. IoT readings flow through an authenticated ingestion layer into a standards-based carbon engine; robust anomaly detection finds wastage within hours; forecasting, benchmarking and scenario modelling turn data into decisions; and role-specific dashboards involve the whole campus community.

On the evaluation campus the system quantified an annualised footprint of about 7,975 $tCO_2e$, detected every injected fault without false alarms, forecast demand within 2.36% error and identified a package of measures worth about 797 $tCO_2e$ per year. Because it is built entirely on free and open-source software and runs on an ordinary computer, it can begin as a low-risk pilot in two or three buildings and grow building by building, exactly as the phased roadmap of the project report recommends.

Beyond its own savings, a monitored campus demonstrates to students and to society how data-driven climate action can work in practice, and the live data strengthens the institution's educational mission.

# REFERENCES


[1] World Resources Institute and WBCSD, *The Greenhouse Gas Protocol: A Corporate Accounting and Reporting Standard*, Revised Edition, 2004.

[2] World Resources Institute, *GHG Protocol Scope 2 Guidance*, 2015.

[3] ISO 14064-1:2018, *Greenhouse gases - Part 1: Specification with guidance at the organization level for quantification and reporting of greenhouse gas emissions and removals*.

[4] ISO 50001:2018, *Energy management systems - Requirements with guidance for use*.

[5] Central Electricity Authority, Government of India, *$CO_2$ Baseline Database for the Indian Power Sector*, User Guide, latest version.

[6] IPCC, *2006 IPCC Guidelines for National Greenhouse Gas Inventories*, Vol. 2: Energy.

[7] IPCC, *Climate Change 2007: The Physical Science Basis* (AR4), global warming potentials.

[8] UK Department for Energy Security and Net Zero, *Greenhouse Gas Reporting: Conversion Factors*.

[9] Bureau of Energy Efficiency, *Star Rating Programme for Office Buildings*, Government of India.

[10] GRIHA Council, *GRIHA Rating System*; Indian Green Building Council, *IGBC Green Campus Rating*.

[11] AASHE, *STARS Technical Manual*, Version 2.2.

[12] United Nations, *Transforming our World: The 2030 Agenda for Sustainable Development*, 2015.

[13] P. J. Rousseeuw and C. Croux, 'Alternatives to the median absolute deviation', *Journal of the American Statistical Association*, 88(424), 1993.

[14] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed., OTexts, 2021.

[15] Django Software Foundation, *Django Documentation*; Encode, *Django REST Framework Documentation*.

[16] Meta Open Source, *React Documentation*; Recharts and Leaflet project documentation.


# APPENDIX: INSTALLATION AND EXECUTION

Requires Python 3.11+ and Node.js 20+ (both free). On Windows, run `setup.bat` once, then `start.bat`, which opens the app at http://localhost:5173; commands for other systems are in README.md. Demo logins: **admin** / admin123, **facility** / facility123, **student** / student123.